AI-asistovaný vývoj

Pravidla pro dlouhodobé a nové projekty

Adaptovatelná šablona — závazná směrnice a best practices

Rámec pro řízení rizik AI-asistovaného vývoje k převzetí a úpravě
Verze v0.1.3 (4f6b057) — 2026-11-07

Obsah

1 Jak tento dokument číst **3**

2 Východisko: proč se pravidla liší podle stáří projektu **3**
2.1 A1. Pravidla platná pro všechny projekty . . . 5
2.2 A2. Směrnice pro dlouhodobé / zavedené projekty . . . 5
2.3 A3. Směrnice pro nové projekty . . . 6
2.4 A4. Matice odpovědnosti podle dosahu škody a typu projektu . . . 6
2.5 A5. Vynucení a revize hranic . . . 7
2.6 A6. Eskalace při porušení . . . 8
2.7 B1. Pro dlouhodobé / zavedené projekty . . . 8
2.8 B2. Pro nové projekty . . . 9
2.9 B3. Společné praktiky . . . 9
2.10 B4. Režimy práce s AI u dat pod NDA . . . 9
2.11 B5. Vrstvená architektura jako podmínka široké AI zóny . . . 10
2.12 B6. Ochrana know-how a odchozí tok dat . . . 11

3 Onboarding checklisty (krok za krokem) **11**
3.1 Checklist 1 — Nasazení AI na zavedený projekt . . . 11
3.2 Checklist 2 — Založení nového projektu s AI . . . 12
3.3 Checklist 3 — Před každým mergem AI-asistovaného MR . . . 12

4 Kategorizace projektu (jak zařadit) **12**
4.1 Zavedený projekt . . . 12
4.2 Hraniční projekt (nový kód v zavedeném kontextu) . . . 12
4.3 Nový projekt (zelená louka) . . . 13

5 Příloha: šablona ADR **13**

6 Rychlá pravidla na závěr **13**

1. Jak tento dokument číst

Toto je šablona, ne hotová politika

Dokument je rámec k převzetí a úpravě, ne směrnice konkrétní organizace. Místa jako „vedení projektu rozhodne“ nebo eskalační autorita jsou záměrně obecná — doplňte je podle své struktury roli. Vezměte si, co sedí; zbytek upravte nebo vypusťte. Šíří se pod licencí CC-BY 4.0 (volné použití včetně komerčního, s uvedením autora).

Co pravidla vynucují a co ne

Pravidla v tomto dokumentu nevynucují úsudek; určují jen místa, kde je úsudek vyžadován, a kdo za něj nese odpovědnost. Mechanika (gate na MR, `CODEOWNERS`, kontroly v CI — viz A5) je vynutitelná, ale sama o sobě nestačí. Jádrem jsou pravidla A1.1 a A1.2, tedy porozumění kódu a jeho převzetí jako vlastního — a ta vynutit nelze, lze je jen vyžadovat v review. To je záměr, ne nedostatek: pravidla jsou psána s úmyslem, aby odpovědnost zůstala na lidech a nepřesunula se na nástroj ani na proces. Pro zavedení rámce z toho plyne, že organizace, která převezme mechaniku bez převzetí odpovědnosti, si nevzala nic — dostane zelené kontroly bez změny rozhodování. Za nasazený je rámec zamýšlen teprve tehdy, když existuje konkrétní jmenovaná osoba ochotná zamítnout MR s odkazem na A1.1 a má k tomu krytí. Pokud taková osoba není, rámec zatím nasazený není, ať je v repozitáři cokoli.

Dokument má dvě jasně oddělené části. Nezaměňuj je — liší se mírou závaznosti.

ČÁST A — Směrnice (závazná)

Pravidla, která MUSÍ být dodržena. Porušení je důvod k zamítnutí MR a v rizikových oblastech může znamenat regulatorní expozici (NIS2/GDPR). **Formulace:** „musí“, „nesmí“, „je zakázáno“.

ČÁST B — Best practices (doporučení)

Postupy, které zvyšují kvalitu a snižují riziko, ale podléhají inženýrskému úsudku. Odchylka je přípustná, je-li vědomá a obhajitelná. **Formulace:** „doporučujeme“, „vhodné“, „zvaž“.

Dvě domény projektů: pravidla se liší podle toho, zda jde o **dlouhodobý/zavedený projekt** (anglicky *legacy* — existuje, běží, má historii), nebo **nový projekt** (zelená louka, hranice se teprve staví).

Hraniční případ

Aktivně rozvíjený zavedený projekt = ZAVEDENÝ. Rozhoduje existence kódové báze a jejího dosahu škody (anglicky *blast radius*), ne intenzita aktuálního vývoje. Nový modul uvnitř zavedeného systému se řídí pravidly pro zavedené projekty, dokud není izolovaný za stabilním kontraktem (rozhraním).

2. Východisko: proč se pravidla liší podle stáří projektu

Rozdíl není svévolný. Vychází ze dvou faktorů, které se mezi zavedeným a novým projektem zásadně liší:

- **Mentální model.** U zavedeného projektu existuje zdokumentovaná i nezdokumentovaná znalost, kterou AI nezná a neumí rekonstruovat ze samotného kódu. Stejně jako AI nezná nezdokumentovaný kontext kódu, nezná ani nezdokumentovaný kontext nad ním — projekt, infrastrukturu, organizaci a vše dál, kam až rozhodnutí sahají. Kód je jen ta nejviditelnější vrstva; pod ní i nad ní leží důvody, které ze samotných artefaktů nevyčteš — proč je něco postavené tak, jak

je, co se kdysi rozbilo, kterých závislostí se nesmí dotknout. U nového projektu se model teprve tvoří — a AI ho může nenápadně nahradit dřív, než vznikne.

- **Dosah škody (blast radius).** Označuje rozsah škody, kterou způsobí chyba v dané části kódu. U zavedeného projektu je už zafixovaný a často velký (integrace, data, závislosti). U nového projektu ho lze ještě navrhnout tak, aby rizikové části byly malé a izolované.

Velikost škody ale není celý obraz rizika. Stejně podstatné je, jak rychle chybu *poznáme* a jak snadno ji *vrátíme*. Riziko dané části kódu proto měříme na třech osách:

- **Dosah škody — jak velká škoda.** Kolik toho selže, když se to pokazí (od lokálního bugu po systémový bezpečnostní incident).
- **Ověřitelnost — jak snadno poznáme, že je to špatně (nebo správně).** Chyba v doménové logice spadne v testu hned; chyba v šifrování se neprojeví v testu, ale jako únik za měsíce. Špatná ověřitelnost = chyba dlouho tiše čeká.
- **Reverzibilita — jak snadno to vrátíme, když se spleteme.** Feature flag nebo vratná migrace sníží riziko i u velkého dosahu; smazaná data bez zálohy jsou nevratná i při malém dosahu.

Osy se kombinují — nejhorší je velký dosah *a* špatná ověřitelnost *a* nevratnost (chyba je velká, dlouho ji nevidíme a nejde vzít zpět). Naopak velký dosah, který je dobře ověřitelný a snadno vratný (feature flag, rychlý rollback), je zvládnutelný. Proto míru přímé kontroly vývojáře (A1.3) neřídí jen velikost škody, ale všechny tři osy dohromady.

Důsledek: u zavedeného projektu chrání pravidla EXISTUJÍCÍ porozumění před erozí. U nových projektů zajišťují, aby porozumění vůbec VZNIKLO a aby se rizikové hranice postavily správně od začátku.

Faktor	Dlouhodobý / zavedený	Nový projekt
Mentální model	Existuje, je cenný, AI ho nezná	Teprve vzniká, AI ho může vytlačit
Dosah škody	Zafixovaný, často velký	Lze navrhnout malý a izolovaný
Hlavní riziko AI	Tichá regrese, narušení zaručených předpokladů	Vývojář bez modelu vlastního kódu
Role pravidel	Chránit porozumění před erozí	Zajistit, že porozumění vznikne
Šíře zóny vedené AI	Užší — jen izolovatelné úkoly	Širší — když skeleton a ADR vznikly dřív

Dosah škody je návrhová proměnná, ne osud

Z tabulky plyne pozitivní důsledek: dosah škody není pevně daný — je do velké míry *výsledkem architektury*. Při dobře vrstveném návrhu, kde je bezpečnostní a datová vrstva izolovaná za stabilními kontrakty, se rizikové jádro smrští na malou, dobře hlídanou plochu. Většina kódu pak leží v zóně nízkého dosahu škody, kde může AI pracovat téměř bez omezení. Architektura tedy není jen ochrana před AI, ale i *nástroj, jak AI bezpečně pustit dál* — rozvedeno v B5.

ČÁST A — Směrnice (závazná)

2.1 A1. Pravidla platná pro všechny projekty

Platí bez ohledu na stáří projektu a mají přednost před vším níže.

A1.1 Pravidlo porozumění. Kód, kterému vývojář nerozumí a nedokáže ho obhájit bez AI, NESMÍ být mergován (začleněn) — bez ohledu na to, že „funguje“.

A1.2 Převzetí jako vlastního. Otázkou není, zda kód napsala AI, ale zda ho vývojář plně převzal jako vlastní. To znamená: rozumět mu, umět vysvětlit jeho trade-offy (kompromisy), doložit jej testy — a v rizikových oblastech i threat modelem (modelem hrozeb) — a nechat jej projít odpovídajícím review. Co tyto podmínky splní, smí být použito i v rizikových oblastech; co je nesplní, nesmí být použito nikde. Výjimkou je správa tajemství (A1.4), kde platí tvrdý zákaz bez ohledu na převzetí.

A1.3 Riziko řídí kontrolu. Čím větší riziko — velikost škody, horší ověřitelnost a horší reverzibilita (viz Východisko) — tím vyšší povinná míra přímé kontroly vývojáře. Klasifikace dle matice v A4.

A1.4 Tajemství a klíče (secrets). AI NESMÍ generovat správu přístupových údajů, klíčů a tokenů (secrets management). Žádné přihlašovací údaje, klíče ani tokeny v kódu či repozitáři — nikdy, v žádném projektu. Zde nejde o porozumění, ale o to, že práce s reálnými tajemstvími přes AI je sama o sobě jejich únikem; proto výjimka z principu A1.2.

A1.5 Označení původu. Kód generovaný AI musí být označen v commitu nebo MR (např. `[ai-generated]`) kvůli auditovatelnosti.

A1.6 Bezpečnostní vrstva — nejvyšší laťka převzetí. U autentizace, autorizace, šifrování, perzistence, tokenů OAuth a auditního logování platí princip A1.2 v nejpřísnější podobě: ať kód navrhne kdokoli, vývojář jej musí plně převzít — doložit threat modelem, testy a obhájit každé rozhodnutí bez AI. Bez tohoto převzetí kód do těchto oblastí nesmí, byť „funguje“.

A1.7 Smlouva o zpracování dat (DPA) před odesláním dat. Žádná osobní data se neposílají na externí AI rozhraní bez platné smlouvy o zpracování (Data Processing Agreement, DPA) — a u služeb mimo EU bez standardních smluvních doložek (Standard Contractual Clauses, SCC). Bez doloženého právního základu pro přenos se nástroj nepoužije; posouzení základu patří roli, která za ochranu osobních údajů odpovídá.

A1.8 Data pod NDA a nedůvěryhodné AI nástroje. AI nástroj, který odesílá kód nebo data ke zpracování třetí straně, se pro účely této směrnice považuje za *přenos dat třetí straně* — bez ohledu na to, že nástroj nabízí volbu „neukládat“ či „neučit se z dat“. Deklarace poskytovatele není ověřitelná záruka. Proto: u dat krytých NDA musí vedení projektu *explicitně rozhodnout o režimu práce s AI* (viz B4) PŘED použitím nástroje. Defaultní režim při absenci rozhodnutí je nejpřísnější (zákaz odeslání).

A1.9 Konfigurace AI nástroje je součástí kódu. Soubory, které řídí chování AI nástroje (pravidla, systémové instrukce, konfigurace vývojového prostředí), podléhají stejnému review jako kód, který ovlivňují, a patří do soupisu rizikových cest dle A5.2. AI je smí navrhnout, ale nesmí je zavést — změna těchto souborů bez převzetí dle A1.2 je porušení směrnice. Kontroluje se i to, co v diffu není na první pohled vidět (neviditelné znaky, změna kódování). Důvod: kdo změní pravidla, změní všechna pravidla najednou — včetně těch, která mají tu změnu zachytit.

2.2 A2. Směrnice pro dlouhodobé / zavedené projekty

Týká se systémů, které jsou v produkci, mají historii a zafixovaný dosah škody.

A2.1 Žádný nepřevzatý zápis do zavedené báze. AI nesmí zapisovat do zavedené báze autonomně — její výstup vstupuje jako návrh, který vývojář převezme dle A1.2 (rozumí, doloží, obhájí), ne jako přímý commit. Tam, kde to vyžaduje citlivost projektu, se výsledky předávají výhradně jako textové instrukce pro vývojové prostředí, nikoli jako hotové soubory.

A2.2 Žádná plošná AI-refaktorizace. Přejmenování integračních vlastností (properties), změny kontraktů a hromadné refaktory se nedělají AI nástrojem nad celou bází. Tento vzor opakovaně vede k vysokým nákladům na nápravu (řádově člověkodny).

A2.3 Změna kontraktu = ADR. Jakákoli změna veřejného kontraktu (API, integrační rozhraní, schéma) vyžaduje záznam architektonického rozhodnutí (Architecture Decision Record, ADR) PŘED implementací, bez ohledu na to, kdo nebo co kód napsal.

A2.4 Reviewer ověřuje zaručené předpoklady. U zavedeného projektu reviewer požaduje vysvětlení, proč změna neporuší předpoklady, na které se spoléhá zbytek systému (a navazující integrace) — ne jen že prošly testy.

A2.5 AI asistuje, vývojář přebírá. Default u zavedené báze je, že AI radí a vývojář výstup přebírá dle A1.2. Režim vedený AI (širší volnost) je výjimka povolená jen pro izolovatelné, snadno ověřitelné úkoly (viz A4).

2.3 A3. Směrnice pro nové projekty

Zelená louka je příležitost postavit hranice správně. Pravidla míří na to, aby AI urychlila start, ale nenahradila tvorbu mentálního modelu a architektury.

A3.1 Skeleton a hranice jako první. Architektonický skeleton (kostru) a hranice dosahu škody (kde leží bezpečnost, data, integrace) definuje vývojář PŘED tím, než AI začne generovat výplň.

A3.2 ADR od commitu č. 1. Klíčová rozhodnutí (perzistence, autentizace, šifrování, hranice modulů) mají ADR od začátku. Nový projekt nemá výmluvu „historicky to tak je“.

A3.3 Izolace rizikových částí. Bezpečnostní a datová vrstva se navrhuje jako malá, izolovaná a testovatelná, aby zóna vedená AI mohla být bezpečně širší.

A3.4 Faktor přežití týmu (bus factor) od začátku. I u nového projektu musí existovat alespoň jedna další osoba schopná kód převzít. AI se do tohoto faktoru nezapočítává.

A3.5 Širší zóna vedená AI jen po splnění výše uvedeného. Teprve když skeleton, ADR a izolace existují, smí AI vést vývoj doménové a UI vrstvy v širším rozsahu než u zavedeného projektu.

2.4 A4. Matice odpovědnosti podle dosahu škody a typu projektu

Sloučená matice: kde AI vede, kde vývojář a kdo je povinný reviewer. Liší se podle domény projektu. „Vede AI“ = AI generuje, vývojář ověřuje; „vede vývojář“ = AI smí asistovat, ale vývojář musí výstup plně převzít dle A1.2 (rozumí, doloží threat modelem a testy, obhájí bez AI). Jediná výjimka je správa tajemství — tam AI nesmí generovat vůbec (A1.4).

Oblast	Dosah	Ov / Rev	Zavedený	Nový projekt	Reviewer
Doménová / business logika	Nízký	+ / +	AI navrhuje, vývojář ověří	Vede AI	Vývojář
UI komponenty, formuláře	Nízký	+ / +	AI navrhuje	Vede AI	Vývojář
Testy, dokumentace	Nízký	+ / +	Vede AI	Vede AI	Vývojář
API endpointy, validace vstupu	Střední	+ / ~	Smíšený, kontrakt ručně	Smíšený	Senior vývojář
Integrační kontrakty / vlastnosti	Stř.–vys.	~ / −	Vede vývojář + ADR	Vede vývojář + ADR	Senior vývojář
Autentizace, autorizace	Vysoký	− / ~	Vede vývojář	Vede vývojář	Senior + Security
Šifrování, klíče	Kritický	− / −	Vede vývojář	Vede vývojář	Senior + Security
Perzistence, migrace DB	Vysoký	~ / −	Vede vývojář	Vede vývojář	Senior vývojář
Správa tajemství (secrets)	Kritický	− / −	AI nesmí generovat	AI nesmí generovat	Senior + Ops
CI/CD, infrastruktura jako kód	Vysoký	~ / +	AI jen rutinní kostra	AI jen rutinní kostra	Senior + Ops

Ov / Rev = ověřitelnost / reverzibilita: + dobrá, ~ částečná, − špatná. Příklad čtení: šifrování má − / − — chybu těžko poznáme i vrátíme, proto vede vývojář bez ohledu na vše ostatní. CI/IaC má velký dosah, ale ~ / + (vratné přes rollback), proto smí AI aspoň rutinní kostru.

Čtení matice: rozdíl zavedený vs nový je nejvýraznější u doménové a UI vrstvy: u nového projektu smí AI vést, u zavedeného jen navrhovat. Bezpečnostní a datová vrstva vede vývojář v obou případech — riziko nezávisí na stáří projektu.

2.5 A5. Vynucení a revize hranic

Pravidla, která nelze vynutit, líný proces obejde. Vynucení stojí na dvou vrstvách: procesní (povinná, funguje i bez silné infrastruktury) a technické (doporučené posílení, zavádí se podle dostupných prostředků). Pravidla jsou *neosobní a symetrická* — platí stejně pro juniora, seniora i architekta. Gate nekontroluje lidi, kontroluje artefakty na hranici.

A5.1 Procesní vrstva (povinná). Veškerá změna codebase jde přes povinný merge request (MR); přímý push do hlavní větve je zakázán. Reviewer MUSÍ odmítnout MR, který nesplňuje Část A — zejména bez vysvětlení designového rozhodnutí, bez ADR u změny kontraktu, nebo s kódem, kterému autor nerozumí. „Funguje to“ není dostatečné zdůvodnění pro merge.

A5.2 Soupis rizikových cest. Cesty (adresáře, moduly) spadající do zóny vedené vývojářem (bezpečnost, data, tajemství, perzistence, CI/IaC) jsou explicitně vyjmenované v repozitáři. Změna v těchto cestách vyžaduje příslušného reviewera dle matice A4 — bez ohledu na velikost změny.

A5.3 Technická vrstva (doporučená, dle prostředků). Tam, kde to infrastruktura dovolí, se

procesní pravidla posílí automaticky: soubor `CODEOWNERS` váže rizikové cesty na povinné reviewery; CI gate (kontrolní úloha) ověří přítomnost ADR u změny kontraktu a označení původu (`[ai-generated]`). Absence této vrstvy NERUŠÍ procesní povinnosti — jen je nenahrazuje automatikou.

A5.4 Periodická revize hranic. Nejen jednotlivé MR, ale i samotné hranice se revidují (doporučeně čtvrtletně): Sedí ještě čára mezi „vede AI“ a „vede vývojář“? Neprosáklo rizikové jádro do běžného kódu (sdílený stav, rozptýlená bezpečnost)? Nenarostl dosah škody v částech, které byly dříve nízkorizikové? Revizi provádí role s přehledem nad architekturou a bezpečností, nezávisle na autorech změn.

A5.5 Zákaz obcházení. Vědomé obejití gate (např. rozdělení rizikové změny na drobné MR, vypnutí kontroly, falešné označení původu) je porušení směrnice samo o sobě — nezávisle na tom, zda výsledný kód „funguje“.

2.6 A6. Eskalace při porušení

Eskalace je odstupňovaná a míří na *nápravu a strukturu*, ne na potrestání osoby. Cílem je vrátit artefakt do souladu s pravidly, ne hledat viníka.

St.	Situace	Reakce	Kdo
1	Neúmyslné porušení	Varování v MR + požadavek na nápravu (vysvětlení, ADR, přepsání). MR se nesloučí, dokud není v souladu.	Reviewer
2	Opakované / nenapravené	Blok MR i větve; změna se nesloučí. Zaznamená se důvod.	Reviewer + senior
3	Úmyslné obcházení gate	Eskalace na autoritu; přezkum dotčených změn zpětně.	[eskalační autorita — doplnit]
4	Únik dat / know-how ven	Okamžité zastavení; řeší se jako bezpečnostní incident (viz B6, NIS2/GDPR).	[eskalační autorita] + Security

- **Default je náprava, ne trest.** Stupně 1–2 jsou běžná součást review; nejsou postihem, ale standardním tlakem na kvalitu.
- **Tvrdší reakce jen u úmyslu a úniku.** Stupně 3–4 se týkají vědomého obcházení a odchozího toku dat — tam, kde je v sázce know-how nebo regulatorní soulad.
- **Eskalační autorita.** V tomto draftu označena obecně jako `[eskalační autorita]`; konkrétní role/osoba se doplní při schválení dokumentu.

ČÁST B — Best practices (doporučení)

Doporučení podléhají úsudku. Odchylka je v pořádku, pokud je vědomá a obhajitelná.

2.7 B1. Pro dlouhodobé / zavedené projekty

- **AI jako oponent (sparring partner).** Nejvyšší přínos AI u zavedeného projektu je v analýze: vysvětlení cizího kódu, hledání dopadů změny, generování hypotéz při ladění — ne v psaní produkčního kódu.
- **Charakterizační testy před zásahem.** Před úpravou nepokrytého místa nech AI navrhnout testy popisující současné chování. Sníží riziko tiché regrese.

- **Inkrementální dávky.** Drobné, oddělené změny s vlastním MR. Velký kódový rozdíl (diff) generovaný AI je u zavedeného projektu těžko zkontrolovatelný a maže mentální model.
- **Doménová intuice nahlas.** Při mentoringu nech juniora vysvětlit, proč je návrh AI v daném kontextu (ne)vhodný — přenos intuice je cennější než samotný kódový rozdíl.

2.8 B2. Pro nové projekty

- **Architektura člověkem, výplň s AI.** Skeleton a rozhraní navrhni sám (klidně v dialogu s lidmi), teprve implementaci vnitřku modulů zrychli s AI.
- **Počítej s driftem (~20 %).** Realizace se od původního návrhu odchýlí. Zachyť odchylku v ADR nebo deníku, ať zůstane vědomá a zdokumentovaná.
- **Procházející kostra (walking skeleton) dřív než šíře.** Nejdřív tenká funkční vertikála přes všechny vrstvy, pak teprve rozšiřování s AI. Ověří hranice dosahu škody v praxi.
- **Tempo vědomě.** Rychlost generování není cíl. U nového projektu je levné zpomalit a postavit hranice správně — drahé je to opravovat později.

2.9 B3. Společné praktiky

- **Lehký ADR proces.** Markdown v repozitáři, ne desetistránkový Word. Zaznamenává proč, alternativy a kompromisy (trade-offy) — viz příloha.
- **Vývojářský deník.** Krátké poznámky o směru a důvodech. Doplňuje ADR o průběžný kontext.
- **Pojmenuj roli.** Vědomě rozlišuj, zda jsi architekt, krizový manažer, konzultant nebo vývojář — a tomu přizpůsob, jak AI používáš.
- **Bez tření = pozor.** Když smyčka s AI nemá tření a vše jen potvrzuje, je do procesu zabudované konfirmační zkreslení. Vlož vědomě kritický krok.

2.10 B4. Režimy práce s AI u dat pod NDA

Toto je oblast, kterou je třeba dořešit — níže uvedené režimy jsou rozhodovací škála, ne hotová politika. Pravidlo A1.8 vyžaduje, aby u dat krytých NDA **vedení projektu zvolilo jeden z režimů** ještě před použitím AI nástroje. Režimy jsou seřazené od nejpřísnějšího k nejvolnějšímu; volba je kompromis mezi přísností a použitelností.

> **Jádro problému:** nástroj typu „AI v editoru“ odesílá obsah ke zpracování na server poskytovatele. Volba „neukládat data“ je deklarace, ne ověřitelná technická záruka — a u dat pod NDA nese odpovědnost za přenos vždy odesílatel.

#	Režim	Podstata	Kompromis
0	Zákaz odeslání	AI se na data pod NDA nepoužije vůbec; jen na nechráněné části.	Nejvyšší jistota, nejnižší přínos AI.
1	Lokální / self-hosted model	Model běží na vlastní infrastruktuře, data neopustí perimetr.	Náklady na provoz a HW; slabší model než cloud.
2	Anonymizace / abstrakce vstupu	Do AI jde jen zobecněný problém bez chráněných identifikátorů a kontextu.	Pracné, riziko zbytkové identifikovatelnosti.
3	Izolovaný sandbox + ruční reflexe	AI vývoj probíhá bokem na neutrálním zadání; výsledek se ručně přenese a přepíše do chráněné báze.	Dvojí práce, ale chráněný kód se nikdy neodešle.

- **Rozhoduje vedení projektu, ne jednotlivý vývojář.** Volba režimu je rozhodnutí s právním dopadem; patří na úroveň, která za NDA odpovídá.
- **Default je režim 0.** Pokud rozhodnutí nepadlo, platí zákaz odeslání.
- **Režim 3 jako prakticky dostupný kompromis.** Izolovaný sandbox s ruční reflexí zpět umožní využít AI a přitom chráněný kód nikdy neopustí perimetr — cenou je dvojí práce a kázeň při přepisu.
- **Kombinace režimů.** Režimy se nevylučují: lze např. anonymizovat (2) vstup do lokálního modelu (1).

2.11 B5. Vrstvená architektura jako podmínka široké AI zóny

Pravidlo A1.3 říká, že míru kontroly určuje riziko. Pozitivní obrácení (naznačené ve Východisku) zní: *dosah škody lze návrhem zmenšit* — a tím legitimně rozšířit prostor, kde smí AI vést. Toto není výmluva pro volnější pravidla, ale návod, jak si volnější pravidla *zasloužit* dobrým návrhem.

> Cíl: stáhnout rizikové jádro (bezpečnost, data, integrace) do malé, izolované a dobře hlídané plochy. Co zůstane venku — doménová logika, UI, transformace — pak leží v zóně nízkého dosahu škody, kde může AI pracovat téměř bez omezení a kde je chyba lokální a snadno ověřitelná.

Co takovou architekturu vytváří:

- **Stabilní kontrakty na hranicích.** Rizikové vrstvy schované za jasným rozhraním (interface), které se nemění s každou změnou logiky. AI pracuje proti kontraktu, ne uvnitř citlivé implementace.
- **Izolace bezpečnostní a datové vrstvy.** Autentizace, šifrování, perzistence a správa tajemství v oddělených, malých modulech, kterých se běžný vývoj nedotýká. Tyto moduly píše vývojář (viz matice A4); zbytek systému je na nich nezávislý.
- **Čisté oddělení vrstev.** Doménová logika neví nic o perzistenci ani transportu. Díky tomu je velká část kódu testovatelná a bezpečně generovatelná, aniž by mohla narušit předpoklady, na kterých rizikové jádro stojí.
- **Hranice dosahu škody jako vědomé rozhodnutí.** Při návrhu se explicitně určí, kudy vede čára mezi „vede AI“ a „vede vývojář“ — a tato čára je zaznamenaná v ADR, ne implicitní.

Hranice tohoto principu

Platí jen tehdy, je-li izolace *skutečná*, ne deklarovaná. Pokud rizikové jádro prosakuje do zbytku systému (sdílený stav, kontrakt měnící se s každou funkcí, bezpečnost rozptýlená napříč kódem), pak „nízký dosah škody“ neexistuje a široká AI zóna je iluze. Široká AI zóna je tedy *odměna za doloženě dobrý návrh*, ne výchozí stav.

2.12 B6. Ochrana know-how a odchozí tok dat

Pravidla v Části A hlídají hlavně *co vstupuje* do codebase. Stejně důležité je *co z ní odchází* — AI nástroj je obousměrný kanál a know-how firmy uniká stejnou trubkou, kterou přitéká pomoc.

> Odeslání kódu nebo kontextu do externího AI nástroje je *odchozí tok dat*. Týká se ho pravidlo A1.7 (DPA), A1.8 (NDA) a režimy B4 — ale i tam, kde nejde o osobní data ani NDA, jde o know-how, jehož hodnota je v tom, že ho konkurence nemá.

- **Co je know-how, ne jen kód.** Doménové modely, integrační vzory, řešení nestandardních problémů, architektonická rozhodnutí — to je hodnota, kterou firma nashromáždila. Odeslání do nástroje, který se z dat učí nebo je uchovává, je tichý odliv této hodnoty.
- **Princip nejmenšího nutného kontextu.** Do AI jde jen tolik kontextu, kolik úkol vyžaduje — ne celý modul „pro jistotu“. Čím méně odchází, tím menší expozice.
- **Citlivé jádro se neodesílá.** Části nesoucí konkurenční výhodu (klíčové algoritmy, doménová specifika) se řeší v režimu, který je neodešle ven (B4 — lokální model, anonymizace, izolovaný sandbox).
- **Symetrie s příchozí stranou.** Stejně jako musí být vstup do rizikového jádra plně převzatý vývojářem (A1.2), nesmí být rizikové jádro nekontrolovaně odesíláno ven. Hranice platí oběma směry.

3. Onboarding checklisty (krok za krokem)

Praktické kroky pro zahájení práce s AI na projektu. Zaškrtni postupně; každý krok musí být splněn před dalším.

3.1 Checklist 1 — Nasazení AI na zavedený projekt

☐ Identifikuj zóny dosahu škody (bezpečnost, data, integrace) a označ je jako vedené vývojářem.

☐ Ověř, že žádné přihlašovací údaje ani klíče nejsou v repozitáři; pokud ano, řeš jako incident před čímkoli dalším.

☐ Zkontroluj existenci DPA/SCC pro každé externí AI rozhraní, kam by mohla téct osobní data.

☐ Pokud data spadají pod NDA, ověř, že vedení projektu zvolilo režim práce s AI dle B4.

☐ Nastav, že výstup AI vstupuje jako návrh nebo textová instrukce, ne jako přímý zápis hotových souborů.

☐ Definuj, které úkoly jsou izolovatelné a smí vést AI (testy, dokumentace, lokální doménová logika).

☐ Zajisti charakterizační testy pro místa, kterých se chystáš dotknout.

☐ Domluv s reviewerem, že u změn kontraktu se vyžaduje ADR a vysvětlení dopadu na zaručené předpoklady.

☐ Ověř faktor přežití týmu: existuje druhá osoba schopná převzít dotčenou část?

3.2 Checklist 2 — Založení nového projektu s AI

☐ Sepiš architektonický skeleton a hranice modulů dřív, než AI cokoli vygeneruje.

☐ Označ na skeletonu rizikové zóny (autentizace, šifrování, perzistence, tajemství) jako vedené vývojářem.

☐ Pokud data spadají pod NDA, ověř zvolený režim práce s AI dle B4.

☐ Založ `docs/adr/` a napiš ADR-0001 pro nejdůležitější rozhodnutí (perzistence/bezpečnost).

☐ Postav procházející kostru — tenkou vertikálu přes všechny vrstvy — ručně nebo s minimální AI asistencí.

☐ Nastav správu tajemství (např. Key Vault) ručně; AI se jí nedotýká.

☐ Připrav kostru CI/CD; AI smí jen rutinní boilerplate, kontroluje Ops.

☐ Teprve teď otevři širší zónu vedenou AI pro doménovou a UI vrstvu.

☐ Zaveď návyk deníku/ADR pro zachycení odchylky od návrhu.

3.3 Checklist 3 — Před každým mergem AI-asistovaného MR

☐ Rozumím každému bloku a dokážu ho obhájit bez AI.

☐ MR nebo commit je označen jako generovaný AI, pokud je.

☐ Změna nezasahuje do zóny vedené vývojářem bez příslušného reviewera.

☐ Pokud se mění kontrakt, existuje ADR.

☐ Žádná tajemství ani klíče v kódovém rozdílu.

☐ U dat pod NDA byl dodržen zvolený režim práce s AI.

☐ Do AI šel jen nutný kontext, ne citlivé jádro nesoucí know-how.

☐ MR neobchází gate (žádné dělení rizikové změny ani falešné označení původu).

☐ Testy popisují chování, ne jen že kód běží.

4. Kategorizace projektu (jak zařadit)

Místo výčtu konkrétních projektů (ten bude předmětem samostatného vzorového dokumentu) zde uvádíme rozhodovací vodítko, do které kategorie projekt patří.

4.1 Zavedený projekt

- **Znaky:** je v produkci, má historii, existující integrace a zafixovaný dosah škody.
- **Režim:** plně se řídí Částí A2 a B1. AI konzultuje, vývojář píše.
- **Pozor:** plošná AI-refaktorizace integračních vlastností bez ADR je typický zdroj nákladné nápravy.

4.2 Hraniční projekt (nový kód v zavedeném kontextu)

- **Znaky:** nově psaný kód, který je ale úzce propojený se zavedeným systémem (sdílené kontrakty, data, integrace).
- **Režim:** dokud není izolovaný za stabilním kontraktem, řídí se pravidly pro ZAVEDENÝ projekt.
- **Příležitost:** jako nový kód má šanci postavit hranice a verzování kontraktů správně od začátku.
- **ADR:** rozhodnutí o tvaru a verzování kontraktů patří do ADR od prvního commitu.

4.3 Nový projekt (zelená louka)

- **Znaky:** bez existující báze a integrací; hranice se teprve staví.
- **Režim:** plný režim Části A3 a B2.
- **Postup:** skeleton → ADR → procházející kostra → izolace rizik → širší zóna vedená AI.

5. Příloha: šablona ADR

ADR (Architecture Decision Record) = krátký záznam architektonického rozhodnutí. Markdown soubor v repozitáři, typicky `docs/adr/0001-nazev.md`. Zaznamenává proč, ne co.

```
# ADR-0001: <název rozhodnutí>

## Status
Přijato - RRRR-MM-DD

## Kontext
Co nás k rozhodnutí vede; relevantní omezení (např. GDPR čl. 32).

## Rozhodnutí
Co jsme zvolili a jak konkrétně.

## Alternativy
- Varianta A - proč ne
- Varianta B - proč ne

## Důsledky
- Závislosti, které vznikají
- Kompromisy, které přijímáme
```

6. Rychlá pravidla na závěr

- **Nevíš přesně, co kód dělá** → nepoužívej.
- **Má to dopad na bezpečnost/data** → smíš použít, jen když to plně převezmeš (A1.2): threat model, testy, obhajoba bez AI. Secrets ani tak (A1.4).
- **Neumíš to vysvětlit bez AI** → nemerguj (nezačleňuj).
- **Zavedená báze** → AI navrhuje, vývojář píše.
- **Nový projekt** → nejdřív skeleton a ADR, pak teprve širší zóna vedená AI.
- **Data pod NDA** → bez rozhodnutí vedení o režimu (B4) se AI nepoužije.
- **Dobře izolované rizikové jádro** → zbytek smí vést AI téměř bez omezení (B5).
- **Vše přes povinný MR** → „funguje to“ není důvod k mergi (A5).
- **Obcházení gate** → porušení samo o sobě, eskaluje se (A5/A6).
- **Know-how je odchozí tok** → citlivé jádro se ven neodesílá (B6).